Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

PAMCA608- SYSTEM DESIGN-

C1+TC1+TCC1

WINTER SEMESTER 2025-26

SCORE

SMART CANTEEN PRE-ORDER SYSTEM

Team:

Mahreen Aisha - 25MCA0079

Sanjana. S - 25MCA0238

Rupali Kumari - 25MCA0217

Deepti Tulsyan - 25MCA0191

Niya Ann Jacob - 25MCA0209

Project Repository: https://github.com/DeeptiTulsyan/smart-canteen-management

Abstract

The Smart Canteen System is a full stack web application developed to streamline the process of browsing menu items and placing food orders within a campus environment. The primary problem addressed by this system is the inefficiency caused by long queues, manual ordering, and delays in food preparation. By enabling users to pre-order meals digitally, the system improves convenience, reduces waiting time, and enhances overall user experience.

The application is built using the MERN stack, which includes MongoDB for database management, Express.js and Node.js for backend services, and React for the frontend interface. To ensure scalability and modularity, the system follows a microservices architecture where different functionalities such as user management, menu handling, order processing, and wallet operations are implemented as independent services.

The system incorporates several key system design concepts. RESTful APIs are used for inter-service communication, enabling seamless data exchange between services. Redis is employed as a caching mechanism to optimize performance by reducing database load, particularly for frequently accessed menu data. RabbitMQ is used for event-driven messaging, allowing asynchronous communication between services such as order processing and notifications. Additionally, the system ensures data consistency and controlled execution through proper validation and structured workflows.

Overall, the Smart Canteen System demonstrates an efficient integration of full stack development with modern system design principles, providing a scalable, high-performance, and user-friendly solution for digital food ordering.

1. Introduction

The Smart Canteen System is designed to address the inefficiencies present in traditional canteen operations, where food ordering is typically handled manually. In such systems, students often experience long queues, delays in service, and lack of real-time information about menu availability. These challenges not only reduce operational efficiency but also lead to a poor user experience. The absence of a streamlined ordering mechanism increases the chances of errors and makes it difficult to manage high demand during peak hours.

To overcome these limitations, there is a need for a digital solution that simplifies and optimizes the food ordering process. The Smart Canteen System provides an online platform where users can browse menu items, place orders in advance, and complete transactions efficiently. This reduces waiting time, improves convenience, and enhances the overall efficiency of canteen management. By automating key processes, the system ensures better accuracy and faster service.

The scope of the application includes providing functionalities for both users and administrators. Users can register, log in, view menu items, add items to their cart, place orders, and manage their wallet balance. Administrators are provided with capabilities to manage menu items by creating, updating, and deleting them. The system also handles order processing and payment operations through dedicated services. Furthermore, the application is designed to be scalable and can be extended to support additional features or multiple canteen environments in the future.

The primary objectives of this system are to reduce waiting time, eliminate long queues, and provide a user-friendly digital platform for food ordering. It also aims to ensure efficient management of menu

items, orders, and payments while maintaining high performance and scalability. The system is built using a microservices architecture to promote modularity and independent service management. Additionally, performance optimization is achieved through caching mechanisms such as Redis, and reliable communication between services is ensured using REST APIs and event-driven messaging.

The application is developed using the MERN stack, which includes MongoDB, Express.js, React, and Node.js. MongoDB is used for storing data in a flexible and scalable manner, while Express.js and Node.js are used to build backend services and APIs. React is used to develop a responsive and interactive frontend interface. By combining these technologies with a microservices approach, the system achieves scalability, maintainability, and efficient performance, making it a robust solution for modern digital food ordering systems.

2. System Requirements Analysis

2.1 Functional Requirements

- User Registration: Users should be able to create an account with valid details.
- User Login: Secure authentication using credentials (JWT-based).
- Menu Viewing: Users can browse available menu items with details like name, price, and category.
- Cart Management: Users can add/remove items to/from the cart.
- Order Placement: Users can place orders by selecting menu items.
- Order Processing: System validates items, calculates total, and updates order status.
- Wallet Management: Users can add balance and make payments using wallet.
- Notification System: Users receive updates about order status (event-driven).
- Admin Controls: Admin can create, update, and delete menu items.
- API Communication: Services communicate using REST APIs.

2.2 Non-Functional Requirements

- Scalability: System supports scaling of individual microservices independently.
- Availability: Services are designed to run independently to avoid complete system failure.
- Performance: Redis caching improves response time and reduces database load.
- Security: JWT authentication ensures secure access and role-based control.
- Reliability: Proper validation and error handling ensure consistent system behavior.
- Fault Tolerance: Use of RabbitMQ enables asynchronous processing and system resilience.
- Maintainability: Microservices architecture allows easy updates and maintenance.
- Responsiveness: Frontend provides a smooth and interactive user experience.

2.3 Assumptions and Constraints

Assumptions:

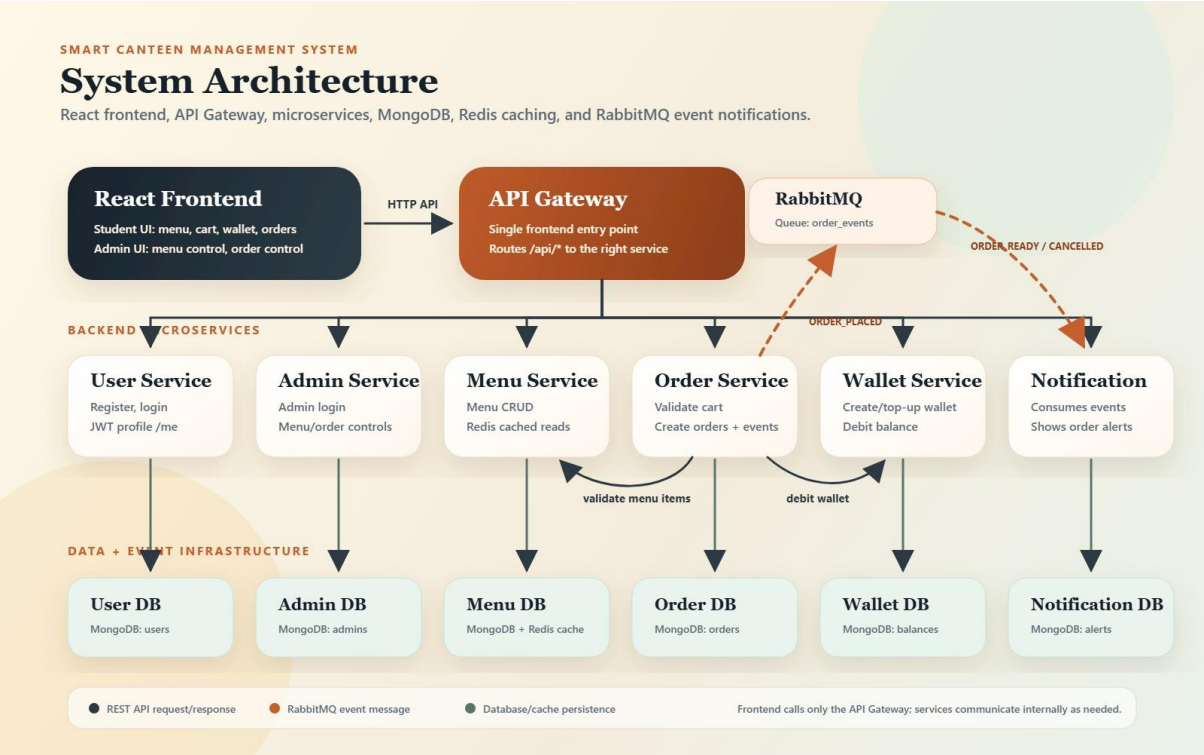
- Users have internet access and basic knowledge to use the system.
- Backend services like MongoDB, Redis, and RabbitMQ are properly configured.
- Users and admins follow proper system usage.

Constraints:

- Dependency on external services like database and message broker.
- System initially designed for limited (campus-level) usage.
- Performance depends on server capacity and network conditions.
- Full functionality requires all services (Redis, RabbitMQ) to be running.

3. System Architecture Design

3.1 Overall Architecture



The system architecture of the Smart Canteen Management System is designed using a microservices-based approach, ensuring modularity, scalability, and independent service management. The frontend of the application is built using React and provides interfaces for both students and administrators. All client requests are routed through an API Gateway, which acts as a single entry point and forwards requests to the appropriate backend services using REST APIs.

The backend consists of multiple microservices, each responsible for a specific functionality. The User Service handles user registration, login, and authentication using JWT. The Admin Service provides administrative controls such as managing menu items and monitoring orders. The Menu Service is responsible for handling menu-related operations, including creating, updating, and retrieving menu items, and it uses Redis caching to improve performance by storing frequently accessed data. The Order Service manages order processing by validating cart items, creating orders, and publishing events. The Wallet Service handles financial transactions such as adding balance and deducting payments during

order placement. The Notification Service listens to system events and provides updates to users regarding their orders.

Each microservice is connected to its own dedicated database, implemented using MongoDB, ensuring data isolation and independence. This design supports better fault isolation and scalability, as each service can operate and scale independently. Redis is integrated with the Menu Service as a caching layer to reduce database load and improve response time.

For asynchronous communication, the system uses RabbitMQ as a message broker. When an order is placed, the Order Service publishes an event (such as "ORDER_PLACED") to RabbitMQ, which is then consumed by other services like the Notification Service. This event-driven approach decouples the services and enhances system flexibility and reliability.

Overall, the architecture combines synchronous communication through REST APIs and asynchronous communication through RabbitMQ, along with caching using Redis, to build a scalable, efficient, and high-performance system.

3.2 Technology Stack

- **Frontend: React**

The frontend of the application is developed using React, which provides a dynamic and responsive user interface. It enables efficient rendering of components such as login, menu, cart, and order pages, and communicates with backend services using REST APIs.

- **Backend: Node.js(with Express.js)**

The backend is built using Node.js and Express.js, which handle business logic, API routing, and inter-service communication. Each microservice (User, Menu, Order, Wallet) is implemented as an independent Node.js service.

- **Database: MongoDB**

MongoDB is used as the primary NoSQL database for storing data such as users, menu items, orders, and wallet information. Each microservice maintains its own database to ensure data isolation and scalability.

- **API Gateway**

The API Gateway acts as a single entry point for all client requests. It routes incoming requests from the frontend to the appropriate microservices and simplifies communication between client and backend services.

- **Message Broker: RabbitMQ**

RabbitMQ is used for event-driven communication between services. It enables asynchronous processing by allowing services to publish and consume events such as order placement and notifications.

- **Cache: Redis**

Redis is used as a caching layer, particularly in the Menu Service, to store frequently accessed data. This reduces database load and improves system performance by providing faster response times.

4. Identification of Microservices and Responsibilities

The Smart Canteen System is designed using a microservices architecture, where each service is responsible for a specific business functionality. This approach ensures modularity, scalability, and independent deployment of services. The key microservices and their responsibilities are as follows:

1. **User Service**

- Handles user registration and login
- Manages user authentication using JWT
- Maintains user profile information
- Provides secure access control for users

2. **Admin Service**

- Handles admin authentication and login
- Provides administrative controls
- Manages menu and order monitoring
- Enables role-based access for admin users

3. **Menu Service**

- Manages menu items (Create, Read, Update, Delete)
- Provides menu data to other services
- Implements Redis caching for frequently accessed data
- Supports filtering (e.g., category-based menu retrieval)

4. **Order Service**

- Handles order creation and processing
- Validates menu items before placing orders
- Calculates total order cost
- Publishes events (e.g., ORDER_PLACED) using RabbitMQ
- Maintains order history and status

5. **Wallet Service**

- Manages user wallet balance
- Handles wallet top-up functionality
- Deducts amount during order placement
- Ensures transaction consistency

6. **Notification Service**

- Listens to events from RabbitMQ
- Sends notifications related to order status
- Provides updates such as order confirmation or cancellation

7. **API Gateway**

- Acts as a single entry point for all client requests
- Routes requests to appropriate microservices
- Simplifies communication between frontend and backend
- Can handle request validation and logging

5. Inter-Service Communication

The Smart Canteen System uses a combination of synchronous and asynchronous communication mechanisms to enable efficient interaction between microservices.

1. **REST APIs (Synchronous Communication)**

- Microservices communicate with each other using RESTful APIs.
- HTTP methods such as GET, POST, PUT, and DELETE are used for data exchange.
- For example, the Order Service calls the Menu Service to validate menu items and retrieve pricing details before placing an order.
- REST APIs ensure real-time communication and immediate response between services.
- They are primarily used for request-response operations where data consistency is required instantly.

2. **API Gateway (Client-to-Service Communication)**

- The API Gateway acts as a single entry point for all client requests.
- The frontend (React application) sends requests to the API Gateway instead of directly calling individual services.

- The API Gateway routes requests to the appropriate microservice based on the endpoint.
- It simplifies client interaction and hides internal service structure.
- It can also handle functionalities such as request routing, authentication validation, and logging.

3. Service-to-Service Communication

- Services communicate directly with each other when required using REST APIs.
- For example: Order Service communicates with Menu Service to validate item details.
- This communication ensures that each service performs its role while collaborating with others.
- Loose coupling is maintained, allowing services to evolve independently.

4. Asynchronous Communication (RabbitMQ)

- The system uses RabbitMQ for event-driven communication between services.
- Services publish events such as "ORDER_PLACED" to the message broker.
- Other services (e.g., Notification Service) consume these events and perform actions asynchronously.
- This reduces dependency between services and improves system scalability and fault tolerance.

6. Concurrency Control and Consistency Handling

Concurrency Control

- The system handles concurrent requests at a basic level using Node.js's asynchronous, non-blocking architecture.
- Since Node.js processes multiple requests concurrently using an event-driven model, the system can handle multiple users accessing services simultaneously.
- For critical operations such as order placement and wallet deduction, requests are processed sequentially within each service to avoid conflicts.
- Although advanced concurrency mechanisms like distributed locking or transactions are not explicitly implemented, the system ensures controlled execution through proper request handling and validation.

Consistency Handling

- Data consistency is maintained by validating all required conditions before performing operations.
- For example, during order placement:
    - The Order Service verifies menu item details (price and availability) from the Menu Service.
    - The Wallet Service checks for sufficient balance before deducting the amount.
- Operations follow a structured flow to ensure correctness before updating the database.
- Cache consistency is maintained by clearing Redis cache whenever menu data is updated (create, update, delete), preventing stale data from being served.
- Event-driven communication using RabbitMQ ensures eventual consistency across services by propagating updates asynchronously.

7. Caching Mechanism

• Use of Redis for Caching

- The system uses Redis as an in-memory caching mechanism to improve performance.
- Redis is integrated specifically with the Menu Service to store frequently accessed menu data.
- Since Redis stores data in memory, it provides much faster access compared to querying the database.

• Caching Strategy

- When a request is made to fetch menu items, the system first checks Redis cache.
- If the data is available in Redis (cache hit), it is returned immediately without querying the database.
- If the data is not available (cache miss), the system fetches it from MongoDB and stores it in Redis for future requests.
- A Time-To-Live (TTL) is set for cached data to ensure automatic expiration after a certain duration.

• Cache Invalidation

- To maintain data consistency, the cache is cleared whenever menu data is modified.
- Operations such as create, update, or delete of menu items trigger cache invalidation.
- This ensures that outdated or stale data is not served to users.

• Performance Benefits

- Reduces the number of database queries, lowering load on MongoDB.
- Improves response time for frequently accessed data.
- Enhances overall system performance and scalability.

• **Example Flow**

- First request → Data fetched from MongoDB and stored in Redis.
- Subsequent requests → Data served directly from Redis cache.
- After update/delete → Cache is cleared, and fresh data is fetched again.

8. Event-Driven Messaging and Asynchronous Processing

Use of Event-Driven Architecture

- The system adopts an event-driven architecture to enable loose coupling between microservices.
- Instead of direct communication for all operations, services communicate through events when appropriate.
- This approach improves scalability, flexibility, and system resilience.

• **Message Broker (RabbitMQ)**

- RabbitMQ is used as the message broker for handling asynchronous communication between services.
- It acts as an intermediary that receives, stores, and forwards messages between producers and consumers.
- Services publish events to RabbitMQ, which are then consumed by other services.

• **Event Publishing**

- The Order Service acts as a producer and publishes events such as "ORDER_PLACED" after successfully creating an order.
- These events contain important details like order ID, items, and total amount.
- Publishing events allows other services to react without direct dependency on the Order Service.

• **Event Consumption**

- Services like the Notification Service act as consumers and listen for events from RabbitMQ.
- Upon receiving an event, they perform specific actions such as sending notifications or updating status.
- This enables asynchronous processing without blocking the main workflow.

• Asynchronous Processing

- Event-driven communication allows services to process tasks in the background without waiting for immediate responses.
- For example, once an order is placed, the system can continue processing notifications independently.
- This improves system performance and reduces response time for users.

9. Database Design

Database Technology

- The system uses **MongoDB**, a NoSQL database, for storing application data.
- MongoDB provides flexibility with schema design and supports scalability for large datasets.
- Mongoose is used as an Object Data Modeling (ODM) library to define schemas and interact with the database.

• Microservices-Based Database Design

- Each microservice maintains its **own independent database**.
- This approach ensures **data isolation, loose coupling, and better scalability**.
- It allows services to function independently without direct dependency on other databases.

• Core Collections (Schemas)

User Collection

- Stores user details such as name, email, password, and role
- Used for authentication and user management

Menu Collection

- Stores menu items with fields like name, price, category, and availability
- Used by Menu Service to manage and retrieve menu data

Order Collection

- Stores order details such as order ID, user ID, items, total amount, and status
- Maintains order history and tracking

Wallet Collection

- Stores user wallet balance and transaction details
- Handles payment-related operations

Notification Collection

- Stores notification messages and status (optional, depending on implementation)

• Data Relationships

- The system uses **logical relationships** rather than strict relational joins.
- For example:
    - Orders reference user ID and menu items
    - Wallet is linked to a specific user
- Data is shared between services through APIs instead of direct database access.

• Data Consistency

- Each service updates only its own database, ensuring **controlled data access**.
- Consistency is maintained through validation and structured workflows across services.

• Advantages of Design

- Supports scalability and high performance
- Enables independent service deployment

- Reduces system complexity through separation of concerns
- Improves fault isolation and maintainability

10. API Design

• Overview of API Design

- The system uses **RESTful API design principles** for communication between frontend and backend services, as well as between microservices.
- APIs are designed using standard HTTP methods such as GET, POST, PUT, and DELETE.
- Each microservice exposes its own set of APIs based on its functionality.

• API Structure

- APIs follow a structured format using resource-based URLs.
- Example:
    - /api/users → User-related operations
    - /api/menu → Menu-related operations
    - /api/orders → Order-related operations
    - /api/wallet → Wallet-related operations

• HTTP Methods Used

- **GET** → Retrieve data (e.g., fetch menu items)
- **POST** → Create new data (e.g., add menu item, place order)
- **PUT** → Update existing data (e.g., update menu item)
- **DELETE** → Remove data (e.g., delete menu item)

• Example APIs (Menu Service)

- GET /api/menu → Fetch all menu items
- GET /api/menu/:id → Fetch a specific menu item
- POST /api/menu → Create a new menu item
- PUT /api/menu/:id → Update a menu item
- DELETE /api/menu/:id → Delete a menu item

• API Communication Flow

- The frontend sends requests to backend services via the API Gateway or directly to service endpoints.
- Backend services process the request and return JSON responses.
- Services may also communicate with each other using APIs for operations like validation and data retrieval.

• Response Format

- APIs return responses in **JSON format**.
- Example:

```
{
  "success": true,
  "data": {...},
  "message": "Request successful"
}
```

• Error Handling

- Proper HTTP status codes are used:
  - 200 → Success
  - 201 → Created
  - 400 → Bad Request
  - 404 → Not Found
  - 500 → Server Error
- Error messages are returned in a structured format for better debugging.

• Security Considerations

- APIs are secured using **JWT authentication**.
- Role-based access control ensures that only authorized users can perform certain operations (e.g., admin-only actions).

The API design follows RESTful principles, ensuring scalability, consistency, and efficient communication between system components.

11. Implementation Details

1. **Frontend (React Components)**

- The frontend is developed using **React**, providing a dynamic and responsive user interface.
- The application is structured using reusable components such as Login, Register, Menu, Cart, Orders, and Admin panels.
- **Context API** is used for managing global state, especially for authentication (AuthContext).
- API calls are handled using **Axios**, enabling communication with backend services.
- Routing is managed to navigate between different pages such as login, menu, and order history.
- The frontend ensures a smooth user experience with real-time data fetching and updates.

2. **Backend (Node.js services)**

- The backend is implemented using **Node.js and Express.js**, following a microservices architecture.
- Each service (User, Menu, Order, Wallet) is developed as an independent module handling specific business logic.
- Controllers manage the core functionality such as handling requests, processing data, and sending responses.
- Routes define API endpoints and connect them to corresponding controller functions.
- Middleware is used for tasks such as authentication, validation, and error handling.
- Redis is integrated within the Menu Service for caching frequently accessed data.
- RabbitMQ is used for event-driven communication between services.

3. **Database (MongoDB collections)**

- The system uses MongoDB as the primary database, with Mongoose for schema modeling.
- Each microservice maintains its own collection to ensure data isolation.
- Key collections include:
    - User Collection → Stores user details and authentication data
    - Menu Collection → Stores menu items and related information
    - Order Collection → Stores order details and status
- Wallet Collection → Stores wallet balance and transactions
- Data is stored in JSON-like documents, allowing flexibility and scalability.
- Database operations include create, read, update, and delete (CRUD) operations handled through Mongoose models.

12. Deployment and Execution

• **Deployment Environment**

- The system is deployed in a **local development environment**.
- Each microservice runs on a separate port using Node.js.
- The frontend runs using Vite development server.
- Required services such as MongoDB, Redis, and RabbitMQ are configured locally.

• **Steps to Run the Application**

- Clone the project repository from GitHub.
- Navigate to each service folder and install dependencies using npm install.
- Configure environment variables (.env) for each service (MongoDB URI, ports, etc.).
- Start backend services using npm run dev or node src/index.js.
- Start the frontend using npm run dev.
- Access the application through the browser (e.g., http://localhost:5173).

• **Service Execution**

- Frontend (React App) runs on port 5173
- User Service runs on port 5001
- Menu Service runs on port 5002
- Order Service runs on port 5003
- Wallet Service runs on port 5004
- Notification Service runs on port 5006

• **System Execution Outputs**

- API Response (Menu Service):

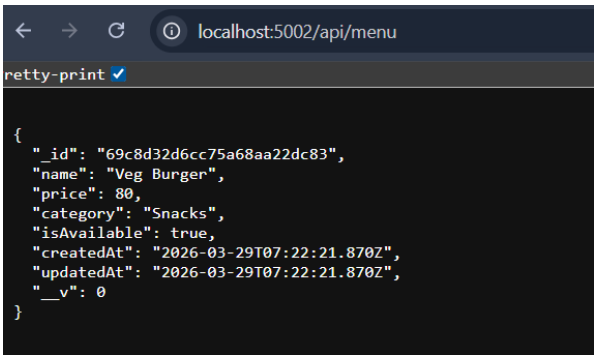


Fig 1: API response showing menu items fetched from Menu Service.

- Terminal shows:

```
Menu Service running on port 5002
Redis connected
MongoDB connected
Data from MongoDB
Data from Redis
Data from Redis
```

Fig 2: Terminal output showing data fetched from MongoDB initially and Redis for subsequent requests.

When the request is made for the first time, data is fetched from MongoDB and stored in Redis cache. For subsequent requests, the data is directly served from Redis, reducing database load and improving performance.

- Event-Driven Processing (Notification Service):

```
◇ injected env (5) from .env // tip: override existing { override: true }
Starting Notification Service...
Connected to DB: mongodb://localhost:27017/notification_db
RabbitMQ connected
Listening to queue...
Notification Service running
Notification HTTP API running on 5006
Event received: ORDER_PLACED
About to save notification: {
  orderId: 'ORD1776259400925',
  studentId: 'deepti123',
  email: 'deeptitulsyan07@gmail.com',
  message: 'Your order is confirmed',
  status: 'PLACED'
}
Saved in DB: {
  orderId: 'ORD1776259400925',
  studentId: 'deepti123',
  email: 'deeptitulsyan07@gmail.com',
  message: 'Your order is confirmed',
  status: 'PLACED',
  _id: new ObjectId('69df917c68f3fbd92df1b0f9'),
  createdAt: 2026-04-15T13:24:12.342Z,
  updatedAt: 2026-04-15T13:24:12.342Z,
  __v: 0
}
Your order is confirmed (Order: ORD1776259400925)
```

Fig 3: Notification Service consuming ORDER_PLACED event from RabbitMQ.

When an order is placed, an event is published to RabbitMQ. The Notification Service listens to the queue, receives the ORDER_PLACED event, processes it, and stores the notification in the database. This demonstrates asynchronous communication between services.

- **Execution Flow**

Step	Component	Action
1	User	Opens the application (web/mobile)
2	User Service	User registers or logs in
3	User Service	Validates email & password using bcrypt and generates JWT
4	Frontend	Receives JWT token
5	Frontend	Stores token and sends in header (Authorization: Bearer <token>)
6	Menu Service	User requests menu items
7	Menu Service	Fetches menu from Redis cache or MongoDB
8	Frontend	User adds items to cart
9	Frontend	Cart managed locally until checkout
10	Order Service	Receives order request with items + JWT
11	Auth Middleware	Extracts and verifies JWT token
12	Auth Middleware	Decodes token and stores user info in req.user
13	Order Controller	Validates order request
14	Order Service	Checks items and quantities
15	Order Service → Menu Service	Fetches actual menu item details
16	Menu Service	Returns name, price, availability
17	Order Service	Builds final order items list
18	Order Service	Calculates total price
19	Order Service	Identifies user via req.user
20	Order Service → Wallet Service	Requests wallet debit
21	Wallet Service	Performs atomic balance update
22	Wallet Service	Rejects if insufficient balance
23	Order Service	Creates order if payment succeeds
24	Order Service	Sets status = **PLACED**
25	Order Service → RabbitMQ	Publishes ORDER_PLACED event

26	RabbitMQ	Stores and delivers event
27	Notification Service	Consumes event asynchronously
28	Notification Service	Sends/logs notification
29	Admin/Kitchen	Updates order status (PREPARING / READY)
30	Order Service	Updates order status in DB
31	Order Service → RabbitMQ	Publishes status update event
32	Notification Service	Notifies user of status change
33	Order Service → Wallet Service	Refunds wallet if order cancelled
34	User	Checks order details via protected routes
35	System Architecture	Uses REST + RabbitMQ + MongoDB + Redis + JWT

13. Conclusion

The Smart Canteen System successfully demonstrates the integration of full stack development with modern system design principles using a microservices architecture. The application effectively addresses the challenges of traditional canteen systems by enabling users to browse menu items, place orders in advance, and manage transactions efficiently through a digital platform. By leveraging the MERN stack, the system ensures a responsive user interface, efficient backend processing, and flexible data storage.

The use of microservices enhances modularity, scalability, and maintainability, allowing each service to operate independently. Key system design concepts such as RESTful APIs for synchronous communication, Redis for caching, and RabbitMQ for event-driven asynchronous messaging contribute to improved performance and system reliability. Additionally, proper validation and structured workflows ensure data consistency across services.

Overall, the project provides a scalable, efficient, and user-friendly solution for digital food ordering while demonstrating practical implementation of real-world architectural patterns. The system can be further extended and enhanced to support larger-scale deployments and additional features in the future.